Studio Nx

P1-22 오류 페이지 연결 — 함수 설명 합본 (백엔드 78~81)

2026-10-12

목차

1. ai-notes/backend(fastAPI_Python)/P1-22 - 오류 페이지 연결/78_errors_is_api_request.md
2. ai-notes/backend(fastAPI_Python)/P1-22 - 오류 페이지 연결/79_errors_error_page.md
3. ai-notes/backend(fastAPI_Python)/P1-22 - 오류 페이지 연결/80_errors_http_error_handler.md
4. ai-notes/backend(fastAPI_Python)/P1-22 - 오류 페이지 연결/81_errors_unexpected_error_handler.md

Backend(FastAPI/Python) 78 — `is_api_request()` (API 요청인지 가르기)

항목	내용
파일	`studionx/backend/app/core/errors.py`
작업	P1-22 오류 페이지 연결: 오류가 났을 때 JSON으로 답할지, 오류 페이지 HTML로 답할지 정하는 첫 단계 (PRD 01 FR-116)
작성	Claude (AI), 2026-10-13, 사용자 검토
이전 · 다음	(P1-22 첫 함수) · 79 `error_page()` (다음)

1. 하는 일

요청의 주소를 보고 **API 요청인지, 화면(페이지) 요청인지** 알려 줍니다. 오류 처리기가 이 결과로 응답 형식을 고릅니다.

요청한 곳	결과	오류가 나면
`fetch('/api/v1/me/dashboard')` 같은 JS의 API 호출	`True`	지금처럼 JSON (`{"error": {...}}`)
브라우저 주소창에 입력한 `/myPage.html`, `/no-such-page`	`False`	오류 페이지 HTML (예: `errors/500.html`)

JS는 JSON의 `error.code`로 분기하므로 API에 HTML을 돌려주면 안 되고, 사람이 주소창으로 연 화면에는 JSON 글자 대신 오류 페이지를 보여 줘야 하기 때문에 둘을 가릅니다.

2. 언제 쓰나

호출하는 곳	용도
80 `http_error_handler` (다음 이후)	404, 405 등 기본 오류에서 HTML과 JSON 중 고르기
81 `unexpected_error_handler`	예상 못 한 500에서 HTML과 JSON 중 고르기

3. 코드

import (이번에 추가한 것만)

```
from app.core.config import get_settings
```

함수

```
def is_api_request(request: Request) -> bool:
    prefix = get_settings().api_prefix
    path = request.url.path
    return path == prefix or path.startswith(prefix + "/")
```

백엔드(Python)는 지금 방식 그대로여서 "최신 문법" 버전은 따로 두지 않습니다.

문법 풀이

식	뜻	예 (`prefix = "/api/v1"`)
`get_settings().api_prefix`	설정에 적힌 API 주소 앞부분입니다. 코드에 `"/api/v1"`을 직접 쓰지 않고 설정을 따릅니다	`"/api/v1"`
`request.url.path`	요청 주소에서 쿼리(`?a=1`)를 뺀 경로입니다	`/home.html?next=/api/v1/me` → `/home.html`
`path == prefix`	API 맨 앞 주소 자체입니다	`/api/v1`
`path.startswith(prefix + "/")`	API 아래의 주소입니다	`/api/v1/me`

4. 흐름

1. 설정에서 API 앞부분을 읽습니다.
2. 요청 경로가 그 앞부분과 같거나, 앞부분 뒤에 `/`가 붙어서 이어지면 `True`입니다.
3. 그 밖의 모든 경로는 `False`입니다.

5. 왜 이렇게

선택	이유
`startswith(prefix)`만 쓰지 않고 `prefix + "/"`로 경계를 확인	`/api/v10`, `/api/v1x` 같은 주소를 API로 잘못 보지 않기 위해서입니다
쿼리 문자열은 보지 않음	`request.url.path`에는 쿼리가 없어서, `/home.html?next=/api/v1/me`가 API로 오해되지 않습니다
설정값(`api_prefix`)을 읽음	`main.py`가 같은 값으로 라우터를 붙이므로 한 곳만 바꾸면 됩니다
헤더(`Accept`)로 가르지 않음	주소로 가르는 쪽이 단순하고, `SiteFiles`(없는 주소 404)도 같은 기준이라 서로 맞습니다
`core/site_files.py`는 그대로	그곳은 `startswith(prefix)`만 쓰는 간단한 판단이며, 이번 작업 범위 밖입니다

6. 시험 (`tests/test_is_api_request.py`, 18개)

시험	결과
API 주소 5개 (`/api/v1`, `/api/v1/`, `/api/v1/me`, `/api/v1/projects/12`, 없는 API)	모두 `True`
화면 주소 7개 (`/`, `/home.html`, `/myPage.html`, `/errors/500.html`, 없는 페이지, `/admin`, `/docs`)	모두 `False`
비슷한 주소 5개 (`/api/v10`, `/api/v1x/me`, `/api`, `/api/v2/me`, `/x/api/v1/me`)	모두 `False`
쿼리에 `/api/v1`이 있는 화면 주소, 쿼리가 붙은 API 주소	`False`, `True`

18개 모두 통과했습니다. 백엔드 전체 시험은 마지막 함수까지 끝낸 뒤 한 번에 돌립니다. `ruff`는 통과했습니다.

Backend(FastAPI/Python) 79 — `error_page()` (오류 페이지 HTML 만들기)

항목	내용
파일	`studionx/backend/app/core/errors.py`
작업	P1-22 오류 페이지 연결: 상태 코드에 맞는 오류 페이지를 같은 상태 코드로 돌려주기 (PRD 01 FR-116)
작성	Claude (AI), 2026-10-13, 사용자 검토
이전 · 다음	78 `is_api_request()` · 80 `http_error_handler` 수정 (다음)

1. 하는 일

상태 코드(404, 500 등)를 받아서, 그 번호의 **오류 페이지 HTML 파일**을 같은 상태 코드의 응답으로 만들어 줍니다. 78번이 "화면 요청"이라고 판단했을 때 오류 처리기가 이 함수를 부릅니다.

받은 상태	보여 주는 페이지	응답 상태
400, 403, 404, 429, 500, 503, 504	같은 번호의 페이지 (`errors/404.html` 등)	그대로
405, 401, 409, 413, 415, 422 같은 나머지 4xx	`errors/400.html`	**원래 번호 그대로**
501, 502, 507 같은 나머지 5xx	`errors/500.html`	**원래 번호 그대로**

2. 언제 쓰나

호출하는 곳	용도
80 `http_error_handler`	화면 요청의 404, 405 등
81 `unexpected_error_handler`	화면 요청의 500

3. 코드

import (이번에 추가한 것만)

```
from fastapi.responses import FileResponse, JSONResponse   # FileResponse 추가
from app.core.config import ROOT_DIR, get_settings          # ROOT_DIR 추가
```

`ERROR_PAGES` 표(번호 → 파일 경로)는 템플릿에 이미 있었습니다.

함수

```
def error_page(status_code: int) -> FileResponse:
    if status_code in ERROR_PAGES:
        page = ERROR_PAGES[status_code]
    elif status_code >= 500:
        page = ERROR_PAGES[500]
    else:
```

```
        page = ERROR_PAGES[400]
    return FileResponse(ROOT_DIR / "frontend" / page, status_code=status_code)
```

백엔드(Python)는 지금 방식 그대로여서 "최신 문법" 버전은 따로 두지 않습니다.

문법 풀이

식	뜻	예
`ERROR_PAGES[status_code]`	표에서 번호로 파일 경로를 찾습니다	`404` → `"errors/404.html"`
`status_code in ERROR_PAGES`	표에 그 번호가 있는지 확인합니다	`405 in ERROR_PAGES` → `False`
`ROOT_DIR / "frontend" / page`	경로 이어 붙이기입니다 (`pathlib`)	`…/studionx/frontend/errors/404.html`
`FileResponse(경로, status_code=…)`	파일 내용을 응답으로 보냅니다. 상태 코드를 직접 정할 수 있습니다	404 페이지를 상태 404로

4. 흐름

1. 상태 코드가 표(`ERROR_PAGES`)에 있으면 그 파일을 고릅니다.
2. 없으면 500 이상은 500 페이지, 그 밖은 400 페이지를 고릅니다.
3. 고른 파일을 **원래 상태 코드로** 응답합니다.

5. 왜 이렇게

선택	이유
상태 코드를 200으로 바꾸지 않고 원래 번호 유지	브라우저, 검색 로봇, 모니터링(Sentry)이 진짜 오류로 인식해야 합니다. 페이지의 큰 숫자와 응답 번호가 같아야 "숫자 자체가 신호"라는 디자인과도 맞습니다
표에 없는 번호는 400, 500 페이지로 대신함	오류 페이지는 7종뿐이라서, 405 같은 번호마다 새 화면을 만들지 않고 가장 가까운 것을 씁니다. 번호가 다른 페이지가 나올 수 있어도 상태는 맞게 나갑니다
5xx는 500 페이지, 나머지는 400 페이지	서버 잘못(5xx)과 요청 잘못(4xx)의 문구가 다르기 때문입니다
파일을 매번 새로 읽음 (`FileResponse`)	오류는 드물어서 부담이 없고, 페이지를 고쳐도 서버를 다시 켜지 않아도 됩니다
표를 코드에 두지 않고 `ERROR_PAGES` 재사용	같은 표를 `core/site_files.py`의 404 처리도 씁니다

6. 시험 (`tests/test_error_page.py`, 17개)

시험	결과
표에 있는 7개 번호 각각	같은 번호의 파일 경로, 상태 코드, `text/html`
표에 없는 4xx 6개 (401, 405, 409, 413, 415, 422)	`errors/400.html`을 쓰고 상태는 원래 번호
표에 없는 5xx 3개 (501, 502, 507)	`errors/500.html`을 쓰고 상태는 원래 번호
7개 페이지 파일이 실제로 있고 자기 번호가 들어 있음	통과입니다

17개 모두 통과했습니다 (78번 18개와 합쳐 35개). `ruff`도 통과했습니다.

Backend(FastAPI/Python) 80 — `http_error_handler()` 수정 (화면은 HTML, API는 JSON)

항목	내용
파일	`studionx/backend/app/core/errors.py`
작업	P1-22 오류 페이지 연결: FastAPI 기본 오류(404, 405 등)를 화면 요청에는 오류 페이지로 응답하기 (PRD 01 FR-116)
작성	Claude (AI), 2026-10-13, 사용자 검토
이전 · 다음	79 `error_page()` · 81 `unexpected_error_handler` 수정 (다음)

1. 하는 일

P1-03에서 만든 처리기에 **화면 요청이면 오류 페이지 HTML**로 응답하는 갈래를 더했습니다. 응답 상태 코드와 `Allow` 같은 헤더는 그대로입니다.

요청	전 (P1-03)	후 (P1-22)
`GET /api/v1/no-such-endpoint` (없는 API)	JSON 404	JSON 404 (그대로)
`POST /api/v1/get-only` (방식이 다름)	JSON 405 + `Allow`	JSON 405 + `Allow` (그대로)
주소창에 입력한 없는 페이지 `/no-such-page`	`SiteFiles`가 404 페이지 (P1-22 이전부터)	그대로
`POST /home.html` (화면 주소에 방식이 다름)	JSON 405	**`errors/400.html`을 상태 405로** (+ `Allow` 헤더)
화면 주소에서 `HTTPException(403)`	JSON 403	**`errors/403.html`을 상태 403으로**
화면 주소에서 `HTTPException(502)`	JSON 502	**`errors/500.html`을 상태 502로**

2. 언제 쓰나

호출하는 곳	용도
`register_exception_handlers()`가 `StarletteHTTPException`에 연결	FastAPI와 Starlette가 직접 낸 오류(없는 주소, 방식이 다름 등)가 날 때마다

3. 코드

import (이번에 추가한 것만)

```
from fastapi.responses import FileResponse, JSONResponse, Response   # Response 추가
```

함수 (바뀐 부분)

```
async def http_error_handler(request: Request, exc: StarletteHTTPException) -> Response:
    if exc.status_code in HTTP_ERRORS:
        code, message = HTTP_ERRORS[exc.status_code]
```

```
    elif exc.status_code >= 500:
        code, message = ErrorCode.INTERNAL_ERROR, "Something went wrong. Please try again."
    else:
        code, message = ErrorCode.BAD_REQUEST, "Bad request."
    if is_api_request(request):
        response = error_json(exc.status_code, code, message)
    else:
        response = error_page(exc.status_code)
    if exc.headers:
        response.headers.update(exc.headers)
    return response
```

바꾼 곳	내용
반환 형식	`JSONResponse`에서 `Response`로 (JSON과 HTML 파일 응답 둘 다 돌려주기 때문입니다)
갈래	`is_api_request(request)` (78)가 참이면 JSON, 거짓이면 `error_page(상태)` (79)
헤더	어느 쪽이든 `exc.headers`를 응답에 붙입니다 (405의 `Allow` 등)

백엔드(Python)는 지금 방식 그대로여서 "최신 문법" 버전은 따로 두지 않습니다.

4. 흐름

1. 오류의 상태 코드로 JSON용 `code`와 `message`를 정합니다 (예전과 같습니다).
2. 요청 주소가 API이면 JSON 응답을 만들고, 화면이면 오류 페이지 응답을 만듭니다.
3. 오류가 가져온 헤더가 있으면 응답에 덧붙입니다.

5. 왜 이렇게

선택	이유
`code`와 `message`를 먼저 구하고 그다음에 갈래	갈래마다 따로 구할 필요가 없고, JSON 쪽의 기존 동작이 한 글자도 바뀌지 않습니다
화면 응답에도 `exc.headers`를 붙임	405 응답은 표준상 `Allow`가 있어야 하며, HTML이어도 같습니다
`exc.detail`을 HTML에 넣지 않음	오류 페이지는 정해진 파일이라서 서버 내부 문구가 섞이지 않습니다
이 처리기만 고치고 `AppError`, 입력 검사 처리기는 그대로	이 두 오류는 `/api/v1`의 라우터 안에서만 나므로, 화면 요청에서는 생기지 않습니다
기존 시험 3개의 주소를 `/api/v1/…`로 옮김	시험용 작은 앱이 `/no-such-api`처럼 API가 아닌 주소를 써서, 이제는 HTML이 돌아오는 것이 맞는 동작이기 때문입니다 (오류 형식을 확인한다는 시험의 목적은 같습니다)

6. 시험

`tests/test_http_error_pages.py` 7개를 새로 만들고, `tests/test_errors.py`는 주소 3곳만 옮겼습니다.

시험	결과
화면 주소의 403	HTML, 상태 403, 본문에 `403`
API 주소의 403	JSON, `error.code`가 `FORBIDDEN`

시험	결과
화면 주소에 POST (405)	HTML(`400.html`), 상태 405, `Allow: GET` 유지
화면 주소의 502	HTML(`500.html`), 상태 502
HTML에 `Traceback`, `HTTPException` 같은 내부 글자가 없음	통과입니다
실제 앱: `POST /home.html`	HTML, 상태 405
실제 앱: 없는 API는 JSON 404, 없는 페이지는 HTML 404	통과입니다

백엔드 pytest 전체 **566개 통과**, `ruff` 통과했습니다.

Backend(FastAPI/Python) 81 — `unexpected_error_handler()` 수정 (500도 화면은 HTML)

항목	내용
파일	`studionx/backend/app/core/errors.py`
작업	P1-22 오류 페이지 연결: 예상 못 한 서버 오류(500)를 화면 요청에는 오류 페이지로 응답하기 (PRD 01 FR-116)
작성	Claude (AI), 2026-10-13, 사용자 검토
이전 · 다음	80 `http_error_handler()` · (P1-22 끝, 실제 화면 시험은 개요 문서에서)

1. 하는 일

버그나 DB 연결 끊김처럼 **예상하지 못한 오류**가 나면 500으로 응답하는 처리기에, 80번과 같은 화면/API 갈래를 더했습니다.

요청	전 (P1-03)	후 (P1-22)
API 주소에서 예상 못 한 오류	JSON 500 `INTERNAL_ERROR`	그대로
화면 주소에서 예상 못 한 오류	JSON 500	**`errors/500.html`을 상태 500으로**

어느 쪽이든 오류의 내용(SQL, 파일 경로, 비밀값이 섞일 수 있는 글자)은 응답에 넣지 않습니다.

2. 언제 쓰나

호출하는 곳	용도
`register_exception_handlers()`가 `Exception`에 연결	다른 처리기가 맡지 않은 모든 오류가 날 때

3. 코드

함수 (바뀐 부분)

```
async def unexpected_error_handler(request: Request, exc: Exception) -> Response:
    if is_api_request(request):
        return error_json(500, ErrorCode.INTERNAL_ERROR, "Something went wrong. Please try again.")
    return error_page(500)
```

바꾼 곳	내용
반환 형식	`JSONResponse`에서 `Response`로 (80번과 같은 이유)
갈래	API이면 예전 그대로 JSON, 화면이면 `error_page(500)`

백엔드(Python)는 지금 방식 그대로여서 "최신 문법" 버전은 따로 두지 않습니다.

4. 흐름

1. 요청 주소를 보고 API인지 판단합니다 (78번).
2. API이면 일반 문장의 JSON 500을, 화면이면 `errors/500.html`을 상태 500으로 돌려줍니다.
3. 응답을 보낸 뒤 Starlette가 원래 오류를 다시 던지므로, 서버 로그와 Sentry에는 원래 오류가 그대로 기록됩니다 (예전과 같습니다).

5. 왜 이렇게

선택	이유
화면에는 `error_page(500)`으로 고정	어떤 버그든 사용자에게는 같은 "서버 문제" 화면을 보여 주고, 내용은 로그로만 남깁니다
오류 내용을 HTML에 넣지 않음	파일 내용이 고정이라서 구조적으로 새지 않습니다
오류를 삼키지 않음	처리기는 응답만 바꾸고, 기록은 Starlette와 Sentry가 맡습니다 (06 설계 그대로)
기존 시험의 주소 `/bug`를 `/api/v1/bug`로 옮김	시험용 앱에서 API 응답(JSON)을 계속 확인하기 위해서입니다

6. 시험

`tests/test_errors.py`의 기존 2개는 주소만 `/api/v1/bug`로 옮겼고, `tests/test_http_error_pages.py`에 2개를 더 했습니다.

시험	결과
API 주소의 오류: 상태 500, 정해진 JSON, `password_hash`와 `RuntimeError` 글자 없음	통과입니다
API 주소의 오류도 응답 뒤에 다시 던져짐 (로그용)	통과입니다
화면 주소의 오류: 상태 500, HTML, `500`이 보임, 내부 글자 없음	통과입니다
화면 주소의 오류도 응답 뒤에 다시 던져짐 (로그용)	통과입니다

백엔드 pytest 전체 **568개 통과**, `ruff` 통과했습니다.